Problem 1

No description has been provided for this image

Part 1

Construct the transformation matrices that describe the kinematics of the three center of mass positions of the links.

In [50]:
```
import sympy as sp
from sympy import Matrix, cos, sin, simplify
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp

# Define Symbols
t  = sp.symbols('t', real=True)
l = sp.symbols('l', real=True)   # Length of Link, assume Links are same length
m = sp.symbols('m', real=True)
k = sp.symbols('k', real=True) # Drag Coefficient
d = sp.symbols('d', real=True)

# Configuration variables as a function of time
x = sp.Function('x')(t)
y = sp.Function('y')(t)
theta = sp.Function('theta')(t)
alpha1 = sp.Function('alpha1')(t)
alpha2 = sp.Function('alpha2')(t)
# Alpha kinematics
omega = sp.symbols("omega",real=True)
A1 = sp.symbols("A1",real=True)
A2 = sp.symbols("A2",real=True)
phi = sp.symbols("phi",real=True)
l = sp.symbols("l", real=True)

q = sp.Matrix([x, y, theta, alpha1, alpha2])
qd = q.diff(t)
qdd = q.diff(t, 2)

# Helper Functions
def cross_matrix(vec):

    return np.array([[0, -vec[2,0], vec[1,0]],
                     [vec[2,0], 0, -vec[0,0]],
                     [-vec[1,0], vec[0,0], 0]])

def uncross_matrix(mat):

    return np.array([[mat[2,1]],
                     [mat[0,2]],
```

```
                    [-mat[0,1]]])

def twist2vec(twist):
    m = Matrix.zeros(6,1)

    m[0:3,0] = uncross_matrix(twist[0:3,0:3])
    m[3:,0] = twist[0:3,3]

    return m

def R_z(theta):
    return np.array([[cos(theta), -sin(theta), 0],
                    [sin(theta),  cos(theta), 0],
                    [0,           0,          1]])

def R_y(theta):
    return np.array([[cos(theta), 0, sin(theta)],
                    [0,          1, 0],
                    [-sin(theta),0, cos(theta)]])

def R_x(theta):
    return np.array([[1, 0, 0],
                    [0, cos(theta), -sin(theta)],
                    [0, sin(theta),  cos(theta)]])

def T(R, p):
    m = Matrix.zeros(4,4)
    m[0:3, 0:3] = R
    m[:3, 3] = p
    m[3, 3] = 1
    return m

# World to body center
T0b = T(R_z(theta), Matrix([[x], [y], [0]]))
T_0b = simplify(T0b)

# Left link COM transform
#T_center_to_left_joint = T(Matrix.eye(3), Matrix([-l/2, 0, 0]))
T_left_joint_rotation  = T(R_z(sp.pi - alpha1), Matrix([-l/2, 0, 0]))
T_left_link_com_offset = T(Matrix.eye(3), Matrix([l/2, 0, 0]))
T_01 = simplify(T_0b @ T_left_joint_rotation @ T_left_link_com_offset)

# Right link COM transform
T_center_to_right_joint = T(Matrix.eye(3), Matrix([l/2, 0, 0]))
T_right_joint_rotation  = T(R_z(alpha2), Matrix([0, 0, 0]))
T_right_link_com_offset = T(Matrix.eye(3), Matrix([l/2, 0, 0]))
T_02 = simplify(T_0b @ T_center_to_right_joint @ T_right_joint_rotation @ T_right_l

print("World to body center transformation matrix:")
T_0b
```

```
World to body center transformation matrix:
```

Out[50]:

$$\begin{bmatrix} \cos(\theta(t)) & -\sin(\theta(t)) & 0 & x(t) \\ \sin(\theta(t)) & \cos(\theta(t)) & 0 & y(t) \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

In [51]:

```
print("Left link COM transformation matrix:")
simplify(T_01)
T_01
```

```
Left link COM transformation matrix:
```

Out[51]:

$$\begin{bmatrix} -\cos(\alpha_1(t)-\theta(t)) & -\sin(\alpha_1(t)-\theta(t)) & 0 & -\frac{l\cos(\alpha_1(t)-\theta(t))}{2} - \frac{l\cos(\theta(t))}{2} + x(t) \\ \sin(\alpha_1(t)-\theta(t)) & -\cos(\alpha_1(t)-\theta(t)) & 0 & \frac{l\sin(\alpha_1(t)-\theta(t))}{2} - \frac{l\sin(\theta(t))}{2} + y(t) \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

In [8]:

```
print("Right link COM transformation matrix:")
T_02
```

```
Right link COM transformation matrix:
```

Out[8]:

$$\begin{bmatrix} \cos(\alpha_2(t)+\theta(t)) & -\sin(\alpha_2(t)+\theta(t)) & 0 & \frac{l\cos(\alpha_2(t)+\theta(t))}{2} + \frac{l\cos(\theta(t))}{2} + x(t) \\ \sin(\alpha_2(t)+\theta(t)) & \cos(\alpha_2(t)+\theta(t)) & 0 & \frac{l\sin(\alpha_2(t)+\theta(t))}{2} + \frac{l\sin(\theta(t))}{2} + y(t) \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Part 2

Generate the body twists for these link center of masses.

In [9]:

```
# Body twist matrices
Vb_hat_1 = simplify(T_01.inv() @ sp.diff(T_01, t))
Vb_hat_b = simplify(T_0b.inv() @ sp.diff(T_0b, t))
Vb_hat_2 = simplify(T_02.inv() @ sp.diff(T_02, t))

# Body twist vectors
Vb_1 = twist2vec(Vb_hat_1)
Vb_b = twist2vec(Vb_hat_b)
Vb_2 = twist2vec(Vb_hat_2)

Vb_b
```

Out[9]:

$$\begin{bmatrix} 0 \\ 0 \\ \frac{d}{dt}\theta(t) \\ \sin(\theta(t))\frac{d}{dt}y(t) + \cos(\theta(t))\frac{d}{dt}x(t) \\ -\sin(\theta(t))\frac{d}{dt}x(t) + \cos(\theta(t))\frac{d}{dt}y(t) \\ 0 \end{bmatrix}$$

In [10]:
```
Vb_1
```

Out[10]:

$$\left[\begin{matrix}0\\0\\-\frac{d}{dt}\alpha_1(t)+\frac{d}{dt}\theta(t)\\\frac{-\frac{l\sin\left(2\alpha_1(t)-\theta(t)\right)\frac{d}{dt}\theta(t)}{2}-\frac{l\sin\left(\theta(t)\right)\frac{d}{dt}\theta(t)}{2}+\sin\left(2\alpha_1(t)-2\theta(t)\right)\frac{d}{dt}y(t)-\cos\left(2\alpha_1(t)-2\theta(t)\right)\frac{d}{dt}x(t)-\frac{d}{dt}x(t)}{2\cos\left(\alpha_1(t)-\theta(t)\right)}\\\frac{l\cos\left(\alpha_1(t)\right)\frac{d}{dt}\theta(t)}{2}-\frac{l\frac{d}{dt}\alpha_1(t)}{2}+\frac{l\frac{d}{dt}\theta(t)}{2}-\sin\left(\alpha_1(t)-\theta(t)\right)\frac{d}{dt}x(t)-\cos\left(\alpha_1(t)-\theta(t)\right)\frac{d}{dt}y(t)\\0\end{matrix}\right]$$

In [11]:
```
Vb_2
```

Out[11]:

$$\left[\begin{matrix}0\\0\\\frac{d}{dt}\alpha_2(t)+\frac{d}{dt}\theta(t)\\\frac{\frac{l\sin\left(2\alpha_2(t)+\theta(t)\right)\frac{d}{dt}\theta(t)}{2}-\frac{l\sin\left(\theta(t)\right)\frac{d}{dt}\theta(t)}{2}+\sin\left(2\alpha_2(t)+2\theta(t)\right)\frac{d}{dt}y(t)+\cos\left(2\alpha_2(t)+2\theta(t)\right)\frac{d}{dt}x(t)+\frac{d}{dt}x(t)}{2\cos\left(\alpha_2(t)+\theta(t)\right)}\\\frac{l\cos\left(\alpha_2(t)\right)\frac{d}{dt}\theta(t)}{2}+\frac{l\frac{d}{dt}\alpha_2(t)}{2}+\frac{l\frac{d}{dt}\theta(t)}{2}-\sin\left(\alpha_2(t)+\theta(t)\right)\frac{d}{dt}x(t)+\cos\left(\alpha_2(t)+\theta(t)\right)\frac{d}{dt}y(t)\\0\end{matrix}\right]$$

Part 3

Generate the body body twist Jacobians that map pose/joint velocities into twists.

In [12]:
```
# Body Jacobians
Jb_1 = simplify(Vb_1.jacobian(qd))
Jb_b = simplify(Vb_b.jacobian(qd))
Jb_2 = simplify(Vb_2.jacobian(qd))

Jb_b
```

Out[12]:

$$\left[\begin{matrix}0&0&0&0&0\\0&0&0&0&0\\0&0&1&0&0\\\cos\left(\theta(t)\right)&\sin\left(\theta(t)\right)&0&0&0\\-\sin\left(\theta(t)\right)&\cos\left(\theta(t)\right)&0&0&0\\0&0&0&0&0\end{matrix}\right]$$

In [13]:
```
Jb_1
```

Out[13]:

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & -1 & 0 \\ -\frac{\cos(2\alpha_1(t)-2\theta(t))+1}{2\cos(\alpha_1(t)-\theta(t))} & \frac{\sin(2\alpha_1(t)-2\theta(t))}{2\cos(\alpha_1(t)-\theta(t))} & -\frac{l\sin(\alpha_1(t))}{2} & 0 & 0 \\ -\sin(\alpha_1(t)-\theta(t)) & -\cos(\alpha_1(t)-\theta(t)) & \frac{l(\cos(\alpha_1(t))+1)}{2} & -\frac{l}{2} & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

In [14]:

```
Jb_2
```

Out[14]:

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ \frac{\cos(2\alpha_2(t)+2\theta(t))+1}{2\cos(\alpha_2(t)+\theta(t))} & \frac{\sin(2\alpha_2(t)+2\theta(t))}{2\cos(\alpha_2(t)+\theta(t))} & \frac{l\sin(\alpha_2(t))}{2} & 0 & 0 \\ -\sin(\alpha_2(t)+\theta(t)) & \cos(\alpha_2(t)+\theta(t)) & \frac{l(\cos(\alpha_2(t))+1)}{2} & 0 & \frac{l}{2} \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Part 4

No description has been provided for this image

In [66]:

```
k, l = sp.symbols('k l', positive=True, real=True)

def body_wrench(Vb, k, l):
    B = Matrix([
        [0, 0, 0,          0,    0,    0],
        [0, 0, 0,          0,    0,    0],
        [0, 0, 2*k*l**3/3,0,    0,    0],
        [0, 0, 0,          k*l,  0,    0],
        [0, 0, 0,          0,   2*k*l, 0],
        [0, 0, 0,          0,    0,    0]
    ])

    Fb = -B * Vb
    return Fb

# Body wrench vectors
Fb_1 = body_wrench(Vb_1, k, l)
Fb_b = body_wrench(Vb_b, k, l)
Fb_2 = body_wrench(Vb_2, k, l)
```

Part 5

In [67]:

```
# Parameters
omega = sp.symbols("omega",real=True)
A1 = sp.symbols("A1",real=True)
A2 = sp.symbols("A2",real=True)
phi = sp.symbols("phi",real=True)
l = sp.symbols("l",real=True)
```

```
params = [l,k,A1,A2,omega,phi]

def Vb(t,q,q_dt,params):
    return -np.linalg.inv(A11_lam(0,q,q_dt,params)) @ A12_lam(0,q,q_dt,params)

def dynamics(t,state,alpha_fcn,params):
    x,y,theta = state
    l, k, A1, A2, omega, phi = params
    alpha1, alpha2, alpha1_dot, alpha2_dot = alpha_fcn(t, omega, A1, A2, phi)
    q_ = [x,y,theta,alpha1,alpha2]
    q_dt_= [0,0,0,0,0]

    dydt= Vb(t,q_,q_dt_,params_) @ np.matrix(([alpha1_dot],[alpha2_dot]))

    return dydt.flatten().tolist()[0]
```

No description has been provided for this image

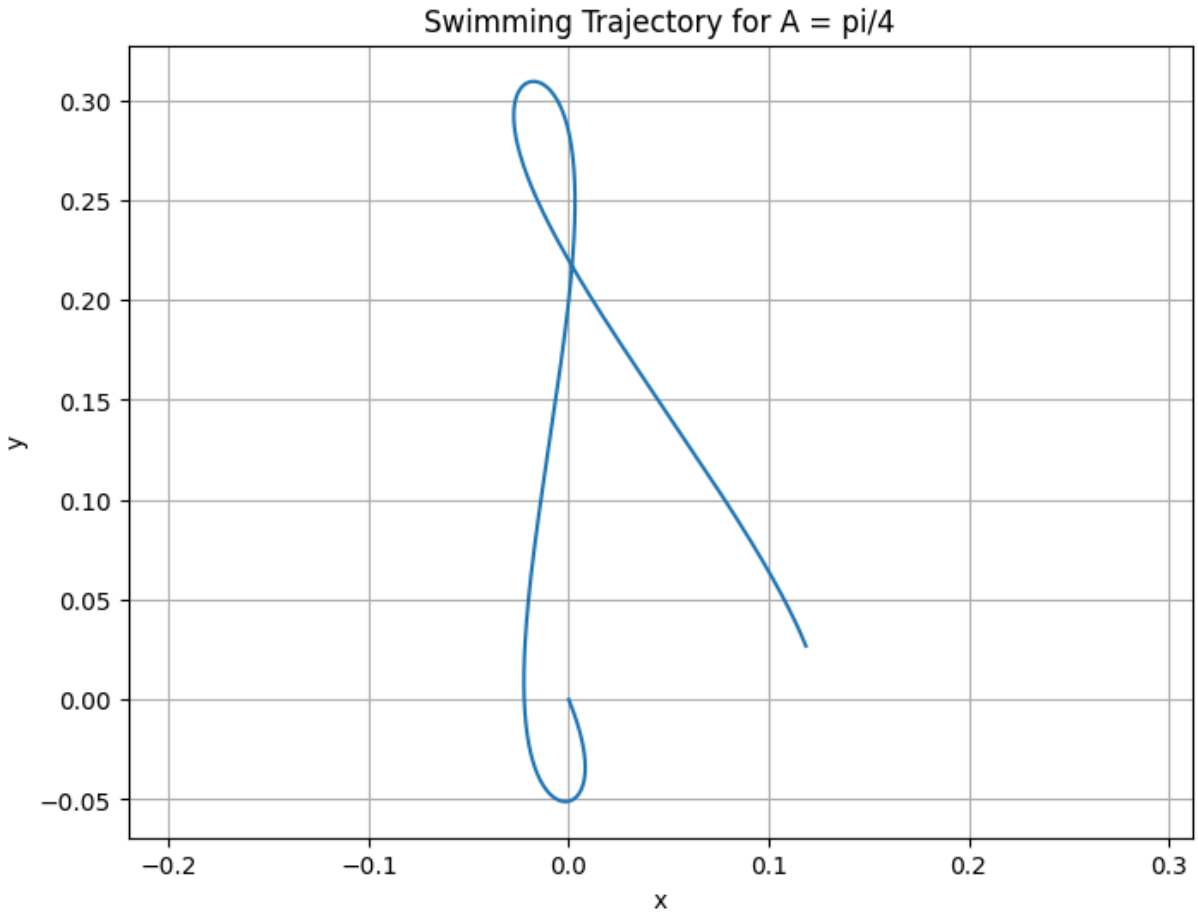
Part 6

In [34]:
```
B = sp.Matrix([
    [0, 0, 0,         0,      0,      0],
    [0, 0, 0,         0,      0,      0],
    [0, 0, 2*k*l**3/3,0,      0,      0],
    [0, 0, 0,         k*l,    0,      0],
    [0, 0, 0,         0,    2*k*l,    0],
    [0, 0, 0,         0,      0,      0]
])

A_mat = (Jb_1.T @ B @ Jb_1) + (Jb_b.T @ B @ Jb_b) + (Jb_2.T @ B @ Jb_2)

A11 = A_mat[:3, :3]
A12 = A_mat[:3, 3:5]

A_tilde = A11.LUsolve(A12)

A11_lam = lambdify([t,q,qd,params],A11)
A12_lam = lambdify([t,q,qd,params],A12)
```

Part 7

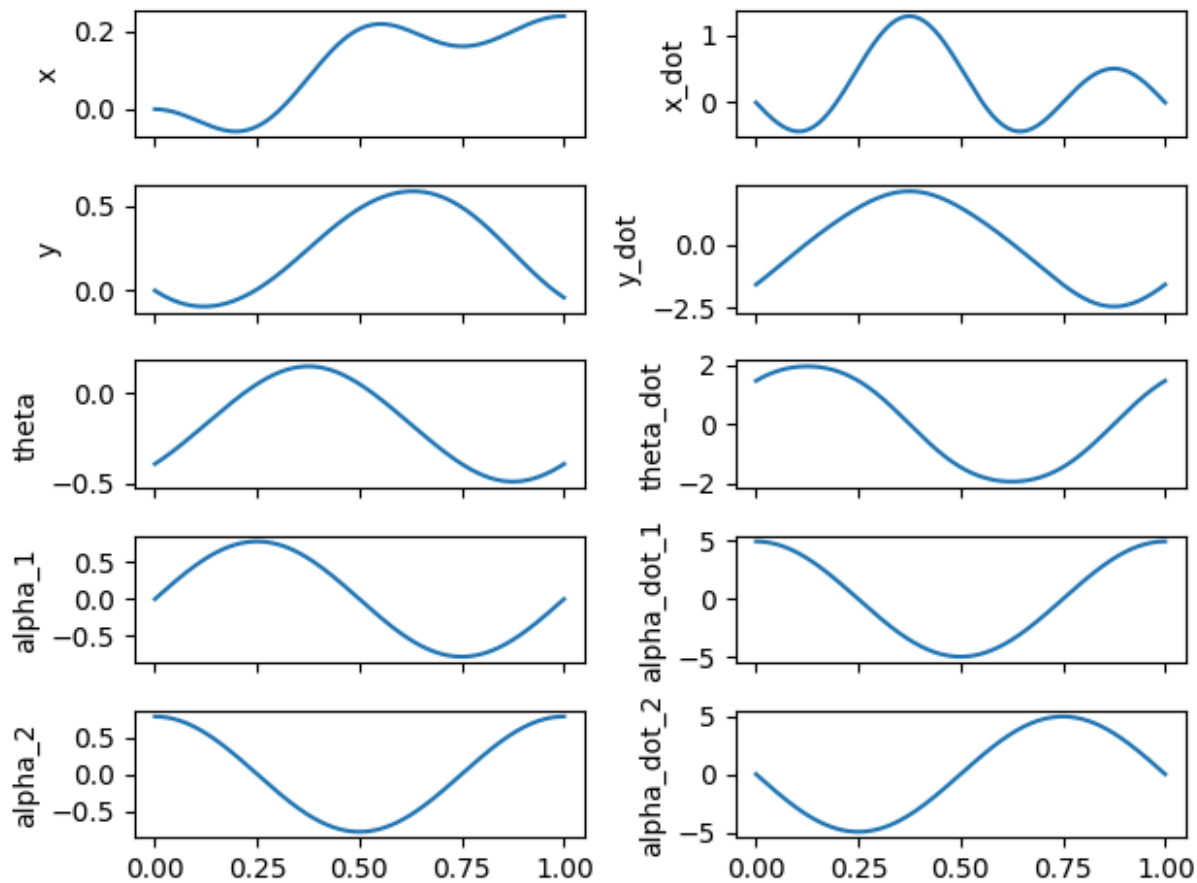
In [35]:
```
from sympy import lambdify
import sympy as sp
from sympy import Matrix, cos, sin, simplify, diff
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp

# Gait Function
alpha_fcn = lambdify((t, omega, A1, A2, phi),[A1*sin(omega*t),
                                A2*sin(omega*t +phi),
```

```
                                    diff(A1*sin(omega*t),t),
                                    diff(A2*sin(omega*t + phi),t)])
l_=1
k_=1
A_amp = np.pi/4
A1_   = A_amp
A2_   = A_amp
phi_  = np.pi/2
omega_= 2*np.pi
params_ = [l_, k_, A1_, A2_, omega_, phi_]

# Initial conditions [x, y, theta]
y0     = [0, 0, 0]
t_span = (0, 1)
t_eval = np.linspace(0, 1, 500)

# Solve
sol = solve_ivp(
    lambda t, y: dynamics(t, y, alpha_fcn, params_),
    t_span,
    y0,
    t_eval=t_eval,
    rtol=1e-8,
    atol=1e-8
)
# Plot x,y trajectory
plt.figure(figsize=(8, 6))
plt.plot(sol.y[0], sol.y[1])
plt.xlabel('x')
plt.ylabel('y')
plt.title('Swimming Trajectory for A = pi/4')
plt.axis('equal')
plt.grid(True)
plt.show()
```



In [39]:
```
# Initial parameters
l_ = 2
w_ = 1
m_ = 1
k_ = 1
A1_ = np.pi/4
A2_ = A1_
omega_ = 2*np.pi
phi_ = np.pi/2
params_ = [l_, k_, A1_, A2_, omega_, phi_]

init_state = [0, 0, -np.pi/8]
t_end = 1
dt = 0.001
time = np.linspace(0, t_end, int(t_end/dt))

sol = solve_ivp(lambda t, y: dynamics(t, y, alpha_fcn, params_),
                [0, t_end], init_state,
                t_eval=time,
                rtol=1e-8, atol=1e-8)

# Fix: unpack params to call alpha_fcn correctly
l_, k_, A1_, A2_, omega_, phi_ = params_
alphas = alpha_fcn(time, omega_, A1_, A2_, phi_)

q_dots = sol.y * 0
for i, tim in enumerate(time):
```

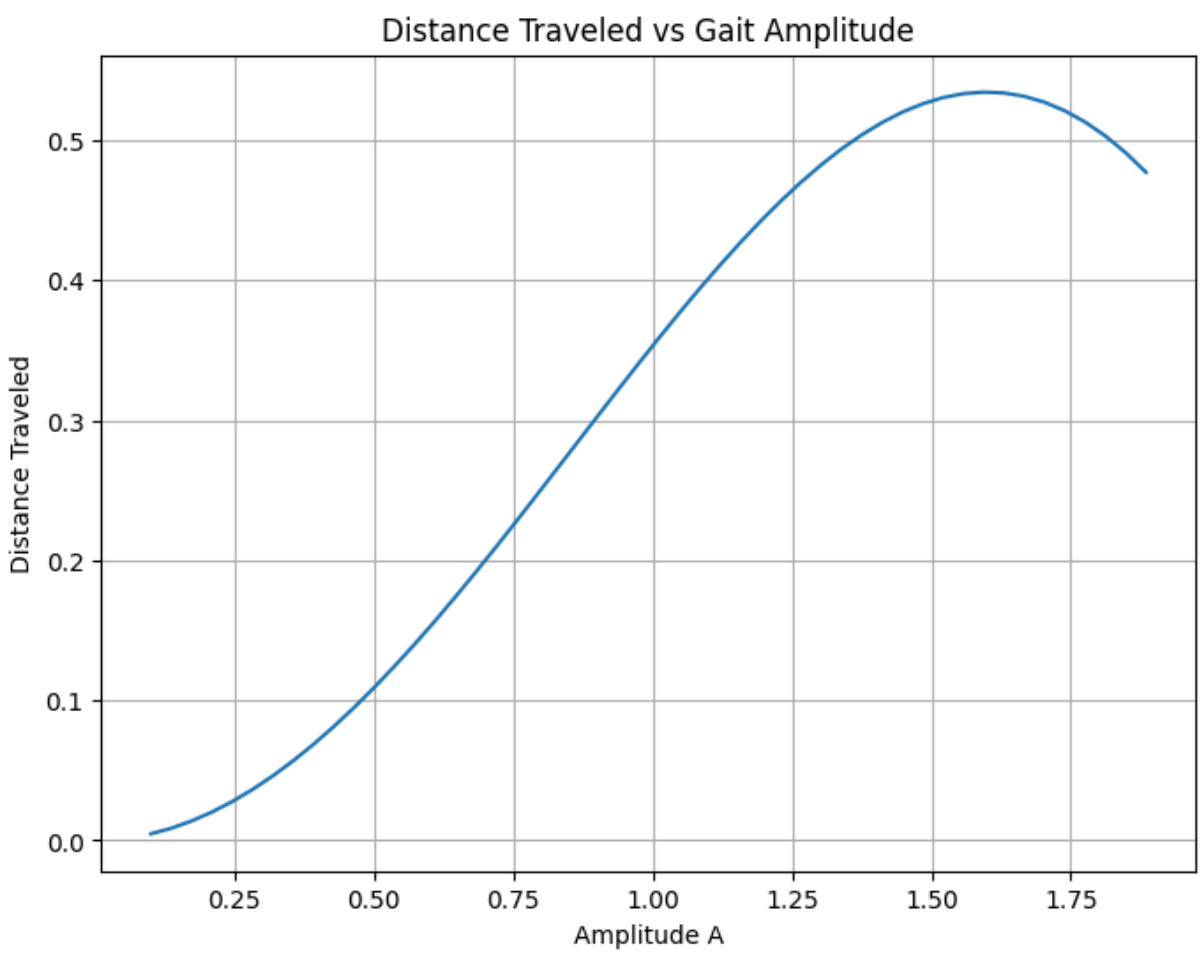
```
    q_temp = dynamics(tim, sol.y[:, i], alpha_fcn, params_)
    q_dots[:, i] = q_temp

x_sol       = sol.y[0, :]
y_sol       = sol.y[1, :]
theta_sol   = sol.y[2, :]
x_dot_sol       = q_dots[0, :]
y_dot_sol       = q_dots[1, :]
theta_dot_sol   = q_dots[2, :]
alpha_0_sol     = alphas[0]
alpha_1_sol     = alphas[1]
alpha_0_dot_sol = alphas[2]
alpha_1_dot_sol = alphas[3]  # Fix: was alphas[2]

fig = plt.figure(2)
fig.clf()
ax = fig.subplots(5, 2, sharex=True)
ax[0][0].plot(time, x_sol);       ax[0][0].set_ylabel('x')
ax[1][0].plot(time, y_sol);       ax[1][0].set_ylabel('y')
ax[2][0].plot(time, theta_sol);   ax[2][0].set_ylabel('theta')
ax[3][0].plot(time, alpha_0_sol); ax[3][0].set_ylabel('alpha_1')
ax[4][0].plot(time, alpha_1_sol); ax[4][0].set_ylabel('alpha_2')
ax[0][1].plot(time, x_dot_sol);       ax[0][1].set_ylabel('x_dot')
ax[1][1].plot(time, y_dot_sol);       ax[1][1].set_ylabel('y_dot')
ax[2][1].plot(time, theta_dot_sol);   ax[2][1].set_ylabel('theta_dot')
ax[3][1].plot(time, alpha_0_dot_sol); ax[3][1].set_ylabel('alpha_dot_1')
ax[4][1].plot(time, alpha_1_dot_sol); ax[4][1].set_ylabel('alpha_dot_2')
plt.tight_layout()
plt.show()
```



Part 8

In [42]:
```
# Part 8 - Sweep Amplitude from 0.1 to 1.2*pi/2
A_vals = np.linspace(0.1, 1.2*np.pi/2, 50)
distances = []

phi_ = np.pi/2

for A1_ in A_vals:

    # Parameters
    A2_ = A1_
    params_ = [l_, k_, A1_, A2_, omega_, phi_]

    # Time setup
    init_state = [0, 0, 0]
    t_end = 1
    dt = 0.01
    time = np.linspace(0, t_end, int(t_end/dt))

    # Solve ODE
    sol = solve_ivp(
        lambda t, y: dynamics(t, y, alpha_fcn, params_),
        (0, t_end),
        init_state,
        t_eval=time,
        rtol=1e-8,
```

```
        atol=1e-8
    )

    # velocities
    q_dots = np.zeros((3, len(time)))

    for i, ti in enumerate(time):
        q_dots[:, i] = dynamics(ti, sol.y[:, i], alpha_fcn, params_)

    # Extract final position
    x_sol = sol.y[0, :]
    y_sol = sol.y[1, :]

    d = np.sqrt(x_sol[-1]**2 + y_sol[-1]**2)
    distances.append(d)
# Plot distance vs amplitude
plt.figure(figsize=(8, 6))
plt.plot(A_vals, distances)
plt.xlabel('Amplitude A')
plt.ylabel('Distance Traveled')
plt.title('Distance Traveled vs Gait Amplitude')
plt.grid(True)
plt.show()
```



Problem 2

Part 1

Write a Python function that takes a body twist vector and returns the body wrench from sliding snake-skin friction.

In [58]:
```
def snake_body_wrench(Vb, f_perp, f_par):
    epsilon = 1E-6
    # Vb is a body twist with wx, wy, wz, vx, vy, vz in body frame
    Fb = np.array([[0],
                   [0],
                   [0],
                   [0],
                   [0],
                   [0]])

    # Check if sliding forward/backward
    if np.abs(Vb[3]) < epsilon:
        Fb[3] = 0
    elif Vb[3] > 0:
        Fb[3] = -f_par
    else:
        Fb[3] = 2*f_par

    if np.abs(Vb[4]) < epsilon:
        Fb[4] = 0
    else:
        Fb[4] = -f_perp*np.sign(Vb[4])

    return Fb
```

#def snake_body_wrench_smooth(Vb, f_perp, f_par): # epsilon = 10 # Vb is a body twist with wx, wy, wz, vx, vy, vz in body frame # Fb = np.array([[0], [0], [0], [0], [0], [0]]) # Fb[3] = -f_par*(1.5*np.tanh(epsilon*Vb[3]+.3)+.3) # Checking if sliding forward/backward # Fb[4] = -f_perp*np.tanh(epsilon*Vb[4]) # return Fb

In [103…
```
def snake_body_wrench_smooth(Vb, f_perp, f_par):
    epsilon = 10

    Vb = np.asarray(Vb, dtype=float).reshape(6, 1)

    Fb = np.zeros((6, 1))

    # Smooth parallel friction
    Fb[3, 0] = -f_par * (1.5 * np.tanh(epsilon * Vb[3, 0] + 0.3) + 0.3)

    # Smooth perpendicular friction
    Fb[4, 0] = -f_perp * np.tanh(epsilon * Vb[4, 0])

    return Fb
```

Part 2

Write a function that takes in the configuration variables of the three-link swimmer and returns the net force from snake-skin sliding friction.

In [104…

```
# Part 2

from sympy import lambdify

q = sp.Matrix([x, y, theta, alpha1, alpha2])
qd = q.diff(t)

params = [l, k, A1, A2, omega, phi]

# Lambdify body twists
v_b_lambda = lambdify([q, qd, params], Vb_b, "numpy")
v_1_lambda = lambdify([q, qd, params], Vb_1, "numpy")
v_2_lambda = lambdify([q, qd, params], Vb_2, "numpy")

# Lambdify body Jacobians
J_b_lambda = lambdify([q, qd, params], Jb_b, "numpy")
J_1_lambda = lambdify([q, qd, params], Jb_1, "numpy")
J_2_lambda = lambdify([q, qd, params], Jb_2, "numpy")


def net_force(q_val, qd_val, params_val, f_par, f_perp):
    """
    q_val = [x, y, theta, alpha1, alpha2]
    qd_val = [xdot, ydot, thetadot, alpha1dot, alpha2dot]
    params_val = [L1, Lb, L2]
    """

    Vb = v_b_lambda(q_val, qd_val, params_val)
    V1 = v_1_lambda(q_val, qd_val, params_val)
    V2 = v_2_lambda(q_val, qd_val, params_val)

    Jb = J_b_lambda(q_val, qd_val, params_val)
    J1 = J_1_lambda(q_val, qd_val, params_val)
    J2 = J_2_lambda(q_val, qd_val, params_val)

    # Convert to numpy arrays
    Vb = np.asarray(Vb, dtype=float).reshape(6, 1)
    V1 = np.asarray(V1, dtype=float).reshape(6, 1)
    V2 = np.asarray(V2, dtype=float).reshape(6, 1)

    Jb = np.asarray(Jb, dtype=float)
    J1 = np.asarray(J1, dtype=float)
    J2 = np.asarray(J2, dtype=float)

    # Important: snake_body_wrench_smooth expects (Vb, f_perp, f_par)
    Fb = snake_body_wrench_smooth(Vb, f_perp, f_par)
    F1 = snake_body_wrench_smooth(V1, f_perp, f_par)
    F2 = snake_body_wrench_smooth(V2, f_perp, f_par)
```

```
    F = Jb.T @ Fb + J1.T @ F1 + J2.T @ F2

    return np.asarray(F, dtype=float).reshape(5)
```

Part 3

In [105…

```
from scipy.optimize import fsolve
import numpy as np

# Numeric parameters
l_val = 1.0
k_val = 1.0

A1_val = np.pi / 4
A2_val = np.pi / 4
omega_val = 2 * np.pi
phi_val = np.pi / 2

params_val = [l_val, k_val, A1_val, A2_val, omega_val, phi_val]

f_par = 1.0
f_perp = 2.0

# Pick one configuration and one shape velocity
q_val = np.array([
    0.0,   # x
    0.0,   # y
    0.0,   # theta
    0.0,   # alpha1
    0.0   # alpha2
])

alpha1dot_val = 1.0
alpha2dot_val = -1.0

def solve_body_velocity(V_guess):
    """
    Unknowns:
        V_guess = [xdot, ydot, thetadot]
    """

    xdot_val, ydot_val, thetadot_val = V_guess

    qd_val = np.array([
        xdot_val,
        ydot_val,
        thetadot_val,
        alpha1dot_val,
        alpha2dot_val
    ])

    F = net_force(q_val, qd_val, params_val, f_par, f_perp)
```

```
    # Force balance for floating body variables x, y, theta
    return F[0:3]

V_solution = fsolve(
    solve_body_velocity,
    x0=np.array([0.0, 0.0, 0.0])
)

print("Solved body velocity [xdot, ydot, thetadot]:")
print(V_solution)
```

```
Solved body velocity [xdot, ydot, thetadot]:
[-1.77335046e-02  0.00000000e+00  8.30623913e+02]
```

```
C:\Users\17076\AppData\Local\Temp\ipykernel_41668\1002490552.py:51: RuntimeWarning:
The iteration is not making good progress, as measured by the
  improvement from the last ten iterations.
  V_solution = fsolve(
```

Part 4

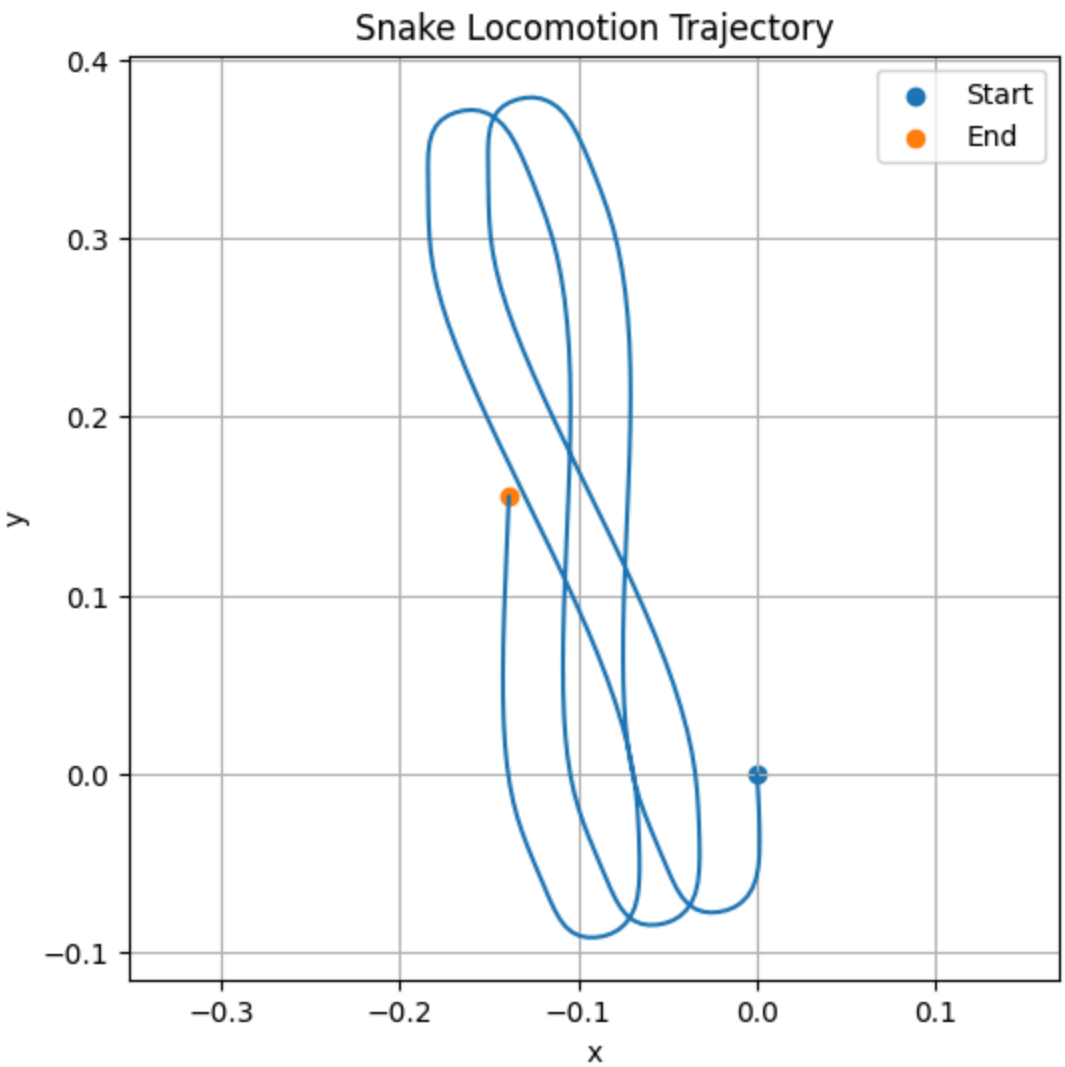
In [108…

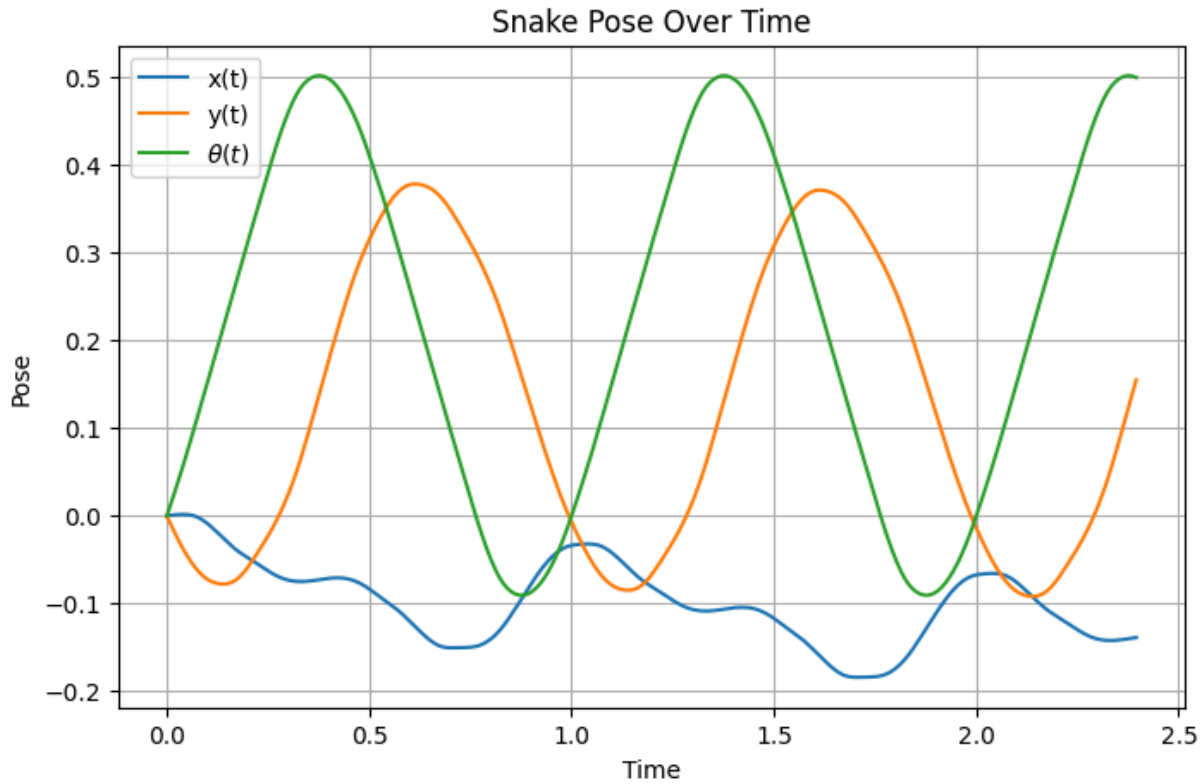
```
# Part 4 dynamics without global variable

from scipy.integrate import solve_ivp
from scipy.optimize import fsolve
import numpy as np
import matplotlib.pyplot as plt

# Store previous fsolve solution inside a list
V_prev = [np.array([0.0, 0.0, 0.0])]

def snake_dynamics(t_num, z):
    x_val, y_val, theta_val = z

    alpha1, alpha2, alpha1dot, alpha2dot = gait(t_num, params_val)

    q_val = np.array([
        x_val,
        y_val,
        theta_val,
        alpha1,
        alpha2
    ], dtype=float)

    def force_balance(V_guess):
        xdot, ydot, thetadot = V_guess

        qd_val = np.array([
            xdot,
            ydot,
            thetadot,
            alpha1dot,
            alpha2dot
        ], dtype=float)

        F = net_force(q_val, qd_val, params_val, f_par, f_perp)
```

```
        return F[0:3]

    # Use the previous solution as the next initial guess
    V_solution = fsolve(force_balance, V_prev[0])

    # Store solution for next timestep
    V_prev[0] = V_solution

    return V_solution

# Initial condition
z0 = np.array([
    0.0,   # x
    0.0,   # y
    0.0    # theta
], dtype=float)

# Time span
num_cycles = 5

omega_val = params_val[4]
period = 2 * np.pi / omega_val

t_span = (0.0, num_cycles * period)
t_eval = np.linspace(t_span[0], t_span[1], 500)

# Run simulation
V_prev[0] = np.array([0.0, 0.0, 0.0])

sol = solve_ivp(
    snake_dynamics,
    t_span,
    z0,
    t_eval=t_eval,
    rtol=1e-6,
    atol=1e-8
)

# Extract solution
x_sol = sol.y[0, :]
y_sol = sol.y[1, :]
theta_sol = sol.y[2, :]

# Plot trajectory
plt.figure(figsize=(6, 6))
plt.plot(x_sol, y_sol)
plt.scatter(x_sol[0], y_sol[0], label="Start")
plt.scatter(x_sol[-1], y_sol[-1], label="End")
plt.xlabel("x")
plt.ylabel("y")
plt.title("Snake Locomotion Trajectory")
plt.axis("equal")
plt.grid(True)
plt.legend()
plt.show()
```

```
# Plot pose over time
plt.figure(figsize=(8, 5))
plt.plot(sol.t, x_sol, label="x(t)")
plt.plot(sol.t, y_sol, label="y(t)")
plt.plot(sol.t, theta_sol, label=r"$\theta(t)$")
plt.xlabel("Time")
plt.ylabel("Pose")
plt.title("Snake Pose Over Time")
plt.grid(True)
plt.legend()
plt.show()
```

```
C:\Users\17076\AppData\Local\Temp\ipykernel_41668\1675699030.py:40: RuntimeWarning:
The iteration is not making good progress, as measured by the
  improvement from the last ten iterations.
  V_solution = fsolve(force_balance, V_prev[0])
```




Snake Pose Over Time
x(t)
y(t)
θ(t)
Pose
Time
0.5
0.4
0.3
0.2
0.1
0.0
−0.1
−0.2
0.0
0.5
1.0
1.5
2.0
2.5

- **Required** Hatton, R. L., & Choset, H. (2013). Geometric Swimming at Low and High Reynolds Numbers. IEEE Transactions on Robotics, 29(3), 615–624.
- **Required** Purcell, E. M. (1977). Life at low Reynolds number. American Journal of Physics, 45(1), 3–11.

Reading 1

For the first required reading I found the content very interesting how low and high Reynolds numbers can be studied with the proper derivation of the geometric mechanics. The paper concluded how with a low Reynolds viscosity dominates the physics used to describe the motion while with a high Reynolds it is inertia dominated.

The paper then explains how using geometric mechanics as discussed in lecture is a framework that connects internal shape changes to net movement of the entire robot. This shows how robots can move without exhibiting a force on the environment by simply changing shape in a cyclic manner which are called gaits. Then, the paper discusses concepts like curvature in order to predict which cyclic motion will create the most displacement. Constraint Curvature Function (CCF) is a very useful geometric tool that acts like a map over the plane which shows where loops create forward motion, backward motion, turning, and efficient/inefficient motion. I really like how the paper shows various graphs and how various gaits can lead to different outcomes. Lastly, the derivation of the reconstruction equation was very interesting even though the notation was slightly different from the lecture. It was very clear and easy to understand the steps in order to parameterize the equations which described how the gait affects the pose variables.

Reading 2

For the second required reading, I found the structure of the paper very interesting. It was kinda like storytelling where the writer talks about the lives of people and how they are doing. I like how it builds an intuitive understanding of motion at low Reynolds numbers. The paper emphasizes that at low Reynolds number viscous forces dominate over inertial forces, meaning that motion on an object behaves very differently from what we are used to expect in everyday life. The main idea of this paper was to show how inertia becomes almost negligible, so objects do not keep moving after you stop applying a force but instead they come to a stop almost immediately. This idea is very interesting because at the time the research paper was written most researchers were focusing on high Reynolds numbers where inertia plays a major role in the dynamics of an object.

Furthermore, this dominance of viscosity leads to constraints of motion for robots and certain organisms that can study how they are able to overcome these viscous forces. For instance, in the “scallop theorem” it shows a simple one degree of freedom motion cannot produce propulsion at low Reynolds numbers. This is very interesting and relates to what we learned in lecture about the importance of non-reciprocal gaits for locomotion.

Additionally, the paper provides examples such as microorganisms like bacteria using flagella to swim. These examples were great because it allowed us to see how motion looks like at this scale on how organisms have to rely on corkscrew-like rotations in order to move rather than just pushing against the

fluid. Overall, I like the sketches used in the paper like the swimmer illustrations. It made the explanation clear and easy to follow.